

From Human-Centric to Machine-Optimized: A Technolinguistic Analysis of Documentation Readability in llm.txt Standards

Yehezkiel Gunawan
Department of Information Systems
Binus University
Jakarta, Indonesia
yehezkiel.gunawan@binus.ac.id

This study presents a technolinguistic analysis comparing human-readable HTML documentation and machine-optimized llm.txt files across ten software engineering documentation platforms. Using established readability metrics (Flesch Reading Ease, Flesch-Kincaid Grade Level, Lexical Density) alongside a novel LLM-as-a-Judge evaluation methodology, we examine the readability paradox inherent in machine-optimized documentation. This analysis reveals that while machine-optimized documentation scores significantly lower on traditional human readability metrics (FRE mean difference: -91.58 points), it receives substantially higher ratings from LLM evaluators (LLM Readability Index mean difference: +14.69 points, $p=0.005$). Large effect sizes (Cohen's d ranging from -1.002 to -1.371) across all LLM evaluation dimensions suggest that machine-optimized documentation is perceived as clearer, more concise, and more LLM-friendly. Crucially, there's no significant correlation between traditional readability metrics and LLM evaluation scores, indicating these constructs measure fundamentally different aspects of documentation quality. These findings provide empirical evidence of the readability-tokenization paradox and suggest that traditional readability metrics are insufficient to evaluate modern documentation standards designed for both human and machine consumption.

Keywords—technolinguistics, documentation readability, LLM-as-a-Judge, llm.txt, Flesch Reading Ease, LLM Readability Index, technical documentation, tokenization

I. INTRODUCTION

The rapid emergence of large language models (LLMs) and artificial intelligence has drastically changed how software engineering documentation is used, analysed, and assessed. Conventional documentation adheres to established readability standards and formatting rules that are optimised for human cognition because it is primarily intended for human readers. But with the growth of machine-optimized documentation standards and LLM-as-a-Judge paradigms, a crucial issue emerges: Does machine-optimized documentation compromise human readability for computer speed, and if so, to what degree?

An important turning point in the development of documentation formats was Jeremy Howard's September 2024 announcement of llm.txt standards. Specifically created for LLM consumption at inference time, this standard suggests a machine-optimized format that offers succinct, expert-level information in a single accessible spot. The fundamental idea is that LLMs gain from better organised, token-efficient

documentation that preserves technical correctness while removing language duplication. The readability conundrum, or the possible conflict between machine-optimized and human-readable documentation, is crucial to this optimisation, though.

Readability has been a cornerstone of technical communication since Rudolf Flesch produced the Flesch Reading Ease formula in 1948, and J. Peter Kincaid created the Flesch-Kincaid Grade Level in 1975. These measures have been extensively accepted in technical documentation, with industry standards suggesting Flesch Reading Ease scores ranging from 60 to 70 for general technical material and 8 to 10 for developer audiences. However, these measures were designed for schoolbooks and general writing, not for the specialised context of machine-optimized documentation.

Text tokenisation, notably Byte Pair Encoding (BPE), developed by Sennrich, Haddow, and Birch in 2016, has become an important concern in documentation design. BPE generates a variable-length encoding in which common words use fewer tokens than unusual or complex terms. This has a direct impact on LLM processing efficiency, as token count correlates with API charges, latency, and context window utilisation. The "Lost in the Middle" phenomenon, observed by Liu et al. (2023), reveals that LLM performance declines when vital information is hidden in extensive contexts, emphasising the importance of compact, front-loaded documentation forms.

Tokenisation of text, notably using Byte Pair Encoding (BPE), developed by Sennrich, Haddow, and Birch in 2016, has become an important concern in documentation design. BPE generates a variable-length encoding, with common words requiring fewer tokens than unusual or complicated terms. This has a direct influence on the efficiency of LLM processing, since token count correlates with API charges, latency, and context window utilisation. The "Lost in the Middle" phenomenon, as demonstrated by Liu et al. (2023), reveals that LLM performance declines when vital information is hidden in extensive contexts, emphasising the need of compact, front-loaded documentation forms.

This work fills a crucial research gap by performing a quantitative computational analysis of human-readable HTML documentation against machine-optimized llm.txt files across ten software engineering documentation platforms. To provide a comprehensive assessment of the readability-tokenization paradox, we use a paired comparison design that controls for content differences by comparing versions of the same

documentation. We also use traditional readability metrics and a novel LLM-as-a-Judge evaluation methodology.

The methodology used in the study is intended to guarantee methodological rigour and repeatability. We created a unique Python CLI tool called readability-python-demo that automates every step of the data collecting process, from site crawling and URL identification to metrics computation and LLM assessment. The contemporary Python package management system uv (Astral), which offers quick dependency resolution and clean environment isolation, is used in the tool's construction. Fifty pytest tests spanning all important components are included in the source, guaranteeing implementation accuracy and making reproducibility easier for other researchers.

Frontend frameworks (React), backend frameworks (FastAPI, Hono), developer platforms (GitHub, Vercel), infrastructure services (Cloudflare), payment systems (Stripe), AI tools (Cursor, LangChain), and backend-as-a-service platforms (Supabase) are among the platforms chosen for analysis. These platforms were chosen in particular because they cater to audiences in software engineering and offer technical documentation on frameworks, deployment platforms, APIs, and developer tools. By ensuring that the results are not restricted to a particular software engineering sub-domain or technology stack, this variety improves the results' generalisability throughout the larger software engineering documentation ecosystem.

Beyond scholarly curiosity, this study is significant. The quality of machine-optimized documentation significantly affects the quality of AI-generated responses as LLM-based AI assistants increasingly mediate developer access to software engineering documentation. Inadequate machine documentation can result in LLM hallucinations, improper use of APIs, or insufficient information retrieval, all of which have an impact on program quality and developer productivity. On the other hand, when working with AI-powered tools, well-optimized machine documentation can boost LLM performance, lower API costs, and improve the developer experience in general.

Given the swift application of AI helpers in software engineering workflows, the study is very pertinent. Technical documentation is now regularly consumed by GitHub Copilot, ChatGPT, Claude, and other AI technologies to offer code recommendations, respond to enquiries, and resolve problems. The calibre of the documentation that the AI consumes has an impact on the calibre of these AI-generated responses. The AI may find it difficult to extract essential information from poorly optimised content, which could result in inaccurate or subpar answers. This paper presents a framework for evaluating the quality of machine-optimized documentation as well as empirical support for its significance.

Additionally, by analysing how documentation formats change to accommodate new customer kinds, the study advances the fields of software engineering and human-computer interaction. Documentation is changing once more to serve AI users, just as it changed from print manuals to HTML web pages to serve online readers. The future of technical communication and the place of human writers in an era of AI-mediated consumption are two significant issues raised by this trend. The study's conclusions lay the groundwork for additional investigation into these issues and offer helpful

advice for documentation teams going through this change.

The development of documentation formats reflects more general changes in information consumption brought about by technology. From the 1960s until the 1990s, print manuals predominated. They had extensive indexes and a table of topics and were intended for linear reading. Hypertext navigation, search capabilities, and multimedia integration were made possible by the switch to HTML-based documentation in the late 1990s, which drastically altered how users interacted with technical information. New assessment techniques were needed for each shift: web documentation introduced measures like click-through rates, time-on-page, and search engine optimisation scores, while print manuals were evaluated using user comprehension exams and job completion durations. Through the development of the LLM Readability Index and the methodical comparison of human and machine assessment metrics, this work introduces yet another paradigm shift in evaluation that is necessary for the present shift to machine-optimized documentation.

This change in documentation has equally important economic consequences. Enterprise documentation teams, which usually consist of dozens of technical writers, editors, and information architects, devote significant resources to the creation and upkeep of software engineering documentation. New cost issues are brought about by the introduction of machine-optimized documentation standards: can a single format satisfy both audiences, or should organisations maintain multiple documentation formats (human-readable and machine-optimized)? The study's conclusions offer information to guide these tactical choices, proving that the readability-tokenization conundrum is a real issue with quantifiable effects on the effectiveness and quality of documentation.

A. Research Questions

This study addresses three primary research questions:

RQ1: Do machine-optimized documentation files (llm.txt) have significantly different traditional readability scores than their human-readable HTML counterparts?

RQ2: Do the LLM-as-a-Judge evaluation scores favour machine-optimized documentation over human-readable HTML documentation?

RQ3: Are existing readability measures (Flesch Reading Ease, Flesch-Kincaid Grade Level) enough for assessing documentation intended for both human and machine consumption, or do they fail to reflect LLM-perceived quality dimensions?

B. Significance of the Study

This study advances the nascent area of technolinguistics by offering empirical evidence for the readability-tokenization conundrum. The findings have ramifications for technical writers, documentation platform developers, and LLM researchers, as they provide insights into the trade-offs between human readability and machine optimisation. In addition, the unique LLM-as-a-Judge approach and LLM Readability Index (LRI) quantification are beneficial instruments for future documentation evaluation study.

II. LITERATURE REVIEW

A. The llm.txt Standard and Machine-Optimized Documentation

In September 2024, Jeremy Howard introduced the llm.txt standard, which marks a paradigm shift in documentation

design [1]. The standard demands a unique Markdown structure that provides brief, expert-level information in a single, easily accessible area, specifically designed for LLM consumption during inference rather than training. According to the official proposal, "Large language models increasingly rely on website information, but face a critical limitation: context windows are too small to handle most websites in their entirety." Converting sophisticated HTML pages containing navigation, advertisements, and JavaScript into LLM-friendly plain text is challenging and imprecise."

Major IT companies like Vercel, Cloudflare, Postman, Cursor, Anthropic, and Hugging Face [2] have adopted llm.txt, as evidenced by the over 2,088 repositories on GitHub that include llm.txt files. The ecosystem has grown beyond the core standard to include several auto-generation tools for VitePress, Docusaurus, and other documentation systems [3], [4], llms-full.txt (expanded versions), and llms-ctx.txt (XML-structured context).

Our research quantifies the tension among human and machine optimisation, which is expressly framed by this norm. Websites are useful for both human readers and LLMs, although the latter gain from having access to more succinct, expert-level information in one convenient spot. The explicit standardisation for machine consumption efficiency is demonstrated by the semantic meaning of the "Optional" section in the llm.txt format, which permits content to be omitted for shorter context.

A H1 title, an H2 summary with succinct professional information, and an optional H2 section with more details that can be omitted for shorter contexts make up the standard llm.txt format. Traditional HTML documentation, that mixes navigation, ads, and interactive features with the real material, stands in stark contrast to this structured approach. The LLM ecosystem has matured with the standardisation of machine-optimized documentation, which recognises that LLMs are active participants who need purpose-built information formats rather than just passive consumers of web content.

The llm.txt adoption trend points to a quick change in business practices. Major internet businesses like Vercel, Cloudflare, and Stripe implemented llm.txt files on their documentation webpages within six months of the standard's proposal. This quick uptake shows that, despite the possible loss of human readability, the technical documentation community values machine-optimized forms. Additionally, the ecosystem has produced automated generation tools that reduce the amount of manual labour needed for implementation by converting current documentation into llm.txt format.

B. Traditional Readability Metrics

The Flesch Reading Ease (FRE) formula, created by Rudolf Flesch in 1948 [5], and the Flesch-Kincaid Grade Level (FKGL), created by J. Peter Kincaid et al. in 1975 [6], are the cornerstones of contemporary readability evaluation. Sentence length and syllable count per word are used by the FRE formula to determine readability, and FKGL converts these metrics into grade level equivalents in the United States.

These criteria have earned academic credibility. FKGL became a US military standard in 1978 after Flesch's initial research was published in the *Journal of Applied Psychology*. The U.S. Department of Defence, Microsoft Word, Grammarly,

and IBM Lotus Symphony have all embraced these measures. However, Redish (2000) [7] and McClure (1987) [8] point out that these formulas have serious drawbacks when used for technical documentation: they were created for school books, not technical content; they ignore reader differences and the effects of content, layout, and retrieval aids; and they assume English text with limited applicability to non-English contexts.

Industry standards for technical documentation suggest grade levels of 8–10 for general developer audiences and FRE scores of 60–70 for general documentation [9]. However, because the subject matter is difficult and the formulae have trouble with polysyllabic technical phrases, technical material inevitably receives lower results. When assessing machine-optimized documentation, which may purposefully utilise dense, succinct language that conventional metrics interpret as less legible, this restriction is very important.

Alternative methods have been developed in response to the shortcomings of conventional readability measurements. While the Automated Readability Index (ARI), Coleman-Liau Index, and Gunning Fog Index provide varying weights for vocabulary difficulty and sentence complexity, they all fundamentally rely on surface-level linguistic traits rather than semantic or pragmatic qualities. Cohesion and coherence measures are incorporated into more contemporary methods like Coh-Metrix, although these instruments are computationally demanding and require specialised software.

It is especially dubious if typical measurements can be used to machine-optimized documentation. Text that has been machine-optimized frequently employs succinct, urgent phrase forms with technical jargon that LLMs can process quickly but humans could find challenging. For instance, the statement "Configure the API key in environment variables" has a low Flesch Reading Ease score because of its complicated noun phrase and technical language, yet LLMs are adept at handling this type of straightforward, practical instruction. The readability-tokenization conundrum examined in this paper is theoretically based on this discrepancy between what conventional metrics measure and what machine users need.

Alternative methods for evaluating readability that might more accurately reflect machine-relevant aspects have been provided by recent developments in computational linguistics. Although the Coleman-Liau Index and the Simple Measure of Gobbledygook (SMOG) provide alternative linguistic complexity weightings, they are still primarily based on human reading comprehension rather than machine processing efficiency. Graesser and McNamara's Coh-Metrix provides a more detailed evaluation of text quality that may partially overlap with machine readability by measuring cohesion and coherence using discourse markers and semantic overlap. Nevertheless, Coh-Metrix is computationally demanding and necessitates certain linguistic processing pipelines that are difficult to implement for extensive documentation analysis.

Neural network-based readability models are a promising approach that connects machine evaluation with conventional measures. These approaches estimate readability scores using pre-trained language models, which may capture contextual and semantic elements missed by formula-based measures. These models, however, have the same human-centric biases that restrict conventional metrics as they were trained on

human readability assessments. The basic problem still exists: readability was established and operationalised for human readers, therefore expanding it to machine consumers necessitates either redefining the concept or creating completely new assessment frameworks. The LLM Readability Index, a complementary metric that reflects machine-perceived quality without claiming to assess the same construct as traditional readability, is proposed in this work to solve this difficulty.

C. Tokenization and Computational Efficiency

The standard tokenisation technique for contemporary LLMs is Byte Pair Encoding (BPE), which was first presented to NLP by Sennrich, Haddow, and Birch in 2016 [10]. BPE is lossless, reversible, and compresses text, usually producing tokens that are about 4 bytes in size each [11]. Variable-length encoding is a crucial feature: ordinary words only need one token, whereas rare or complicated words need several. Recent research has examined how tokenisation choices impact model behaviour [12], and the selection of subword merge operations has a substantial impact on downstream performance [13].

Documentation design is significantly impacted by the token economy. Token count is directly correlated with API cost and delay, according to research, and well-structured, token-efficient prompts outperform verbose prompts by 10–15% while costing 5–10 times less. The necessity for succinct, front-loaded documentation formats like llm.txt [14] is directly motivated by the empirical evidence of the “Lost in the Middle” phenomena (Liu et al., 2023), which shows that LLMs struggle to use information in the middle of lengthy contexts.

Published at EMNLP 2023, LLMLingua (Jiang et al., 2023) demonstrates that well-structured compression may retain semantic integrity while dramatically lowering token count [15], achieving up to 20x prompt compression with minimal performance loss. Nevertheless, this study concentrates on compression methods rather than documentation standards, which leaves a vacuum in our knowledge of how choices in documentation format impact machine efficiency and human readability.

D. LLM-as-a-Judge Evaluation Methodology

One methodological advancement in documentation assessment is the employment of LLMs as assessors. Recent research has shown that LLMs can offer dependable, consistent assessments of text quality in a variety of contexts. This approach provides a machine-centric view of documentation quality that supplements conventional human-focused metrics by utilising the intrinsic capacity of LLMs to evaluate clarity, completeness, conciseness, and technical correctness.

The LLM-as-a-Judge approach is particularly relevant for evaluating machine-optimized documentation, as it captures dimensions that traditional readability metrics may miss. While Flesch Reading Ease measures sentence length and syllable complexity, LLM evaluators can assess semantic coherence, information density, and LLM-specific optimization factors such as token efficiency and context window utilization. Recent studies by Zheng et al. [16] have demonstrated that strong LLM judges can achieve over 80% agreement with human preferences, making LLM-as-a-Judge a scalable and explainable evaluation methodology. This approach has been successfully applied to code readability evaluation, as shown by Horikawa et al. [17] and Simoes et al. [18], who

demonstrated that LLMs can effectively evaluate semantic quality aspects of technical documentation. Additionally, Diggs et al. [19] explored LLM-generated documentation for legacy code systems, highlighting both opportunities and limitations in automated documentation generation.

E. Research Gap

Despite the extensive literature on readability metrics, tokenization efficiency, and LLM evaluation, a significant research gap remains: no peer-reviewed study has systematically compared traditional readability metrics with LLM evaluation scores for machine-optimized documentation. This study addresses that gap by providing a quantitative analysis of the readability-tokenization paradox across ten technical documentation platforms.

III. METHOD

A. Research Design

This study employs a quantitative content analysis approach with a paired comparison design. We compare human-readable HTML documentation and machine-optimized llm.txt files for the same platforms, controlling for content differences by analyzing versions of the same documentation. The methodology combines traditional computational linguistics metrics with a novel LLM-as-a-Judge evaluation system.

B. Corpus Selection

The corpus comprises ten software engineering documentation platforms selected for their adoption of llm.txt standards and their representation of diverse technical domains within the software engineering discipline. The selection criteria included: (1) presence of both human-readable HTML documentation and machine-optimized llm.txt files, (2) technical documentation focus (excluding marketing sites and blogs), (3) English language content, and (4) accessibility via public URLs.

TABLE I: Platform Characteristics

Platform	Category	Domain
React	UI Framework	Frontend
GitHub	Developer Platform	DevOps
Supabase	Backend-as-a-Service	Backend
Vercel	Deployment Platform	DevOps
Stripe	Payment Platform	Fintech
LangChain	AI Orchestration	AI/ML
Cloudflare	Infrastructure	DevOps
Cursor	AI Editor	Developer Tools
Hono	Web Framework	Backend
FastAPI	Python Framework	Backend

C. Data Collection

Data collection was performed using a custom Python CLI tool (readability-python-demo) developed specifically for this study [20]. The tool was built using the modern Python package management system `uv` (Astral), which provides rapid dependency resolution, clean environment isolation, and reproducible builds across different computing environments. The architecture follows a modular design pattern with clear separation of concerns between data acquisition, processing, evaluation, and export components.

The tool implements a comprehensive five-step pipeline for automated documentation auditing, as illustrated in Figure 1.

Each step was designed to ensure methodological rigor, data quality, and reproducibility. The pipeline processes platforms sequentially but supports resumable execution through caching mechanisms, allowing researchers to resume interrupted batches without data loss.

1) *Automated Detection Mechanism*: The first step involves automated detection of machine-optimized documentation standards. The tool performs asynchronous HTTP GET requests to each platform’s root domain, checking for the presence of `/llm.txt` and `/llms.txt` files. This detection mechanism uses the `httpx` library with configurable timeouts and retry logic to handle network variability and server response delays.

The detection logic follows a hierarchical approach: (1) check for `/llms.txt` first, as this is the newer and more comprehensive standard, (2) fall back to `/llm.txt` if the first path is not found, (3) verify that the response returns HTTP status code 200 with content type `text/plain` to ensure the file is actually readable text and not an error page or redirect. Platforms that do not return a valid response are flagged as `NOT FOUND` and skipped from further analysis, implementing the purposive sampling strategy described in the corpus selection criteria.

For platforms where machine-optimized documentation is detected, the system downloads the complete text content and stores it as a raw text file for subsequent processing. The tool also handles relative links within the `llm.txt` files, automatically following references to `llms-full.txt` or other linked documentation files to ensure comprehensive content retrieval.

2) *Deep Web Crawling for Human Documentation*: For human-readable documentation, the tool employs the Crawl4AI framework to perform breadth-first search (BFS) multi-page crawling with configurable depth and page limits. Crawl4AI uses Playwright for browser automation, enabling JavaScript rendering for modern single-page applications (SPAs) such as Cursor documentation, which requires dynamic content loading.

The crawling strategy is configured with the following parameters: (1) maximum depth of 3 levels to prevent infinite crawling, (2) page limit of 50 pages per platform to balance comprehensiveness and computational efficiency, (3) semantic content extraction using DOM structure analysis, paragraph density heuristics, and link distribution patterns to distinguish content from navigation elements. The scraper extracts the main content area while discarding sidebars, footers, headers, and other peripheral content.

The extracted content is converted to clean Markdown format, removing HTML tags and preserving the document structure. This Markdown representation serves as the basis for traditional readability metrics calculation, as it provides a clean text representation without the visual formatting overhead of HTML.

3) *Text Cleaning and Prose Extraction*: To ensure fair comparison between human-readable HTML and machine-optimized text, the tool implements a sophisticated text cleaning pipeline. Human-readable HTML documentation undergoes prose extraction using Mozilla Readability [21], a well-established library for extracting article content from web pages.

The cleaning process removes the following elements: (1) code blocks and inline code snippets, which artificially inflate technical density metrics, (2) navigation elements, breadcrumbs, and sidebar menus, which add structural text without semantic content, (3) images, advertisements, and promotional content, (4) table of contents and index pages, (5) inline styles and scripts. The pipeline preserves paragraph text, link anchor text, and semantic headings, as these represent the actual content that human readers consume.

A minimum content threshold of 500 characters is enforced; if the cleaned text falls below this threshold, the tool attempts alternative extraction strategies or flags the platform for manual review. This ensures that readability metrics are calculated on substantial content rather than sparse fragments.

4) *Linked Content Aggregation for Machine Documentation*: For machine-optimized documentation, the tool goes beyond simple file download by following all linked files referenced in the main `llm.txt` file. The aggregation mechanism: (1) parses the Markdown content of `llm.txt` to identify links, (2) resolves relative URLs to absolute URLs, (3) downloads all linked `.txt` files, (4) concatenates the content in the order specified by the links, (5) handles `llms-full.txt` and `llms-ctx.txt` variants where available.

This linked aggregation ensures that the machine documentation corpus is comprehensive and representative of the full documentation available to LLMs. The tool also performs length comparison between the aggregated content and the original `llm.txt` to verify that the expanded version is indeed more comprehensive, as expected.

5) *Context7 API Fallback Strategy*: For platforms where `llm.txt` is not directly available, the tool implements a Context7 API fallback mechanism. Context7 provides structured documentation content for many open-source libraries and frameworks. The fallback logic follows a three-tier approach: (1) if `llm.txt` is found but does not reference `llms-full.txt`, query Context7 for additional content, (2) if Context7 returns more content than the original `llm.txt`, use the Context7 version as it is more comprehensive, (3) if `llm.txt` is not found at all, use Context7 as the primary source.

This fallback strategy ensures that the study includes platforms that have adopted machine-optimized documentation standards indirectly, through third-party aggregation services. The Context7 API requires authentication via a token stored in environment variables, ensuring secure access without exposing credentials in the codebase.

6) *LLM Evaluation with Caching and Rate Limiting*: The LLM-as-a-Judge evaluation component employs a robust asynchronous architecture with automatic retry logic and exponential backoff. The system uses `httpx.AsyncClient` for concurrent API requests to the OpenRouter API [22], with the following resilience mechanisms:

(1)	Response	Caching:	All	LLM
evaluation	responses	are	cached	to
				<code>results/llm_cache/domain_doc_type_chunk.json</code>

files. This enables resumable batch processing; if the API fails mid-batch (e.g., due to rate limiting or network issues), re-running the tool will skip cached evaluations and only process missing items.

(2) **Rate Limiting:** The tool automatically handles HTTP 429 (Too Many Requests) responses by respecting the `Retry-After` header and implementing exponential backoff. The free tier of OpenRouter has strict rate limits, and the tool is designed to work within these constraints.

(3) **Content Chunking:** Long documentation texts are split into manageable chunks (approximately 4,000 tokens each) to stay within the context window limits of the evaluation model. Each chunk is evaluated independently, and the scores are averaged across chunks.

(4) **Prompt Engineering:** The evaluation uses structured prompts with JSON output format requirements, ensuring consistent and parseable responses. The prompt template includes: (a) the documentation text to evaluate, (b) the five evaluation dimensions with detailed rubrics, (c) instructions for scoring on the 1-5 Likert scale, (d) JSON schema for the response format.

The evaluation model, `meta-llama/llama-3.2-3b-instruct`, was selected for its balance of performance and accessibility. The model is available on OpenRouter’s free tier, making the methodology reproducible for other researchers without financial barriers. The 3B parameter size is sufficient for documentation evaluation tasks while being computationally efficient.

Step 1: Detection. The tool automatically detects `llm.txt` and `llms.txt` files at the root path of each platform using asynchronous HTTP requests.

Step 2: Deep Crawling. For human-readable documentation, the tool performs BFS multi-page crawling using `Crawl4AI`, with configurable depth and page limits. The scraper extracts semantic content by analyzing DOM structure, paragraph density, and link distribution.

Step 3: Text Cleaning. Human-readable HTML documentation undergoes prose extraction using `Mozilla Readability` [21] to ensure fair comparison. The cleaning pipeline removes code blocks, navigation elements, inline code, images, and advertisements while preserving paragraph text and link anchor text.

Step 4: Linked Content Aggregation. For machine-optimized documentation, the tool follows links in `llm.txt` files to fetch complete content, including `llms-full.txt` where available.

Step 5: Context7 Fallback. For platforms without direct `llm.txt` files, the Context7 API is used as a fallback, with length comparison to ensure comprehensive content retrieval.

D. Metrics and Instruments

1) **Traditional Readability Metrics:** The study employs three traditional readability metrics implemented using the Python `textstat` library:

Flesch Reading Ease (FRE): Calculated as $206.835 - 1.015(\text{total words}/\text{total sentences}) - 84.6(\text{total syllables}/\text{total words})$. Scores range from 0 to 100, with higher scores indicating easier readability.

Flesch-Kincaid Grade Level (FKGL): Calculated as $0.39(\text{total words}/\text{total sentences}) + 11.8(\text{total syllables}/\text{total words}) - 15.59$. Scores represent U.S. grade level equivalents.

Lexical Density (LD): Calculated as the percentage of content words (nouns, verbs, adjectives, adverbs) relative to

total words, using the NLTK stopwords corpus for stopword identification.

Token-to-Word Ratio (TWR): Calculated using OpenAI’s `tiktoken` library with the `cl100k_base` encoding (GPT-4). This metric measures the ratio of tokens to words, indicating tokenization efficiency.

2) **LLM-as-a-Judge Evaluation:** The LLM evaluation employs a five-dimension Likert scale assessment via the OpenRouter API [22], using the `meta-llama/llama-3.2-3b-instruct` model. The evaluation dimensions are:

- 1) **Clarity** (1-5): Is the documentation clear and unambiguous?
- 2) **Completeness** (1-5): Does it cover the topic adequately?
- 3) **Conciseness** (1-5): Is it free of unnecessary verbosity?
- 4) **Technical Accuracy** (1-5): Are technical details correct?
- 5) **LLM-Friendliness** (1-5): Is it optimized for machine consumption?

LLM Readability Index (LRI): To enable direct comparison with Flesch Reading Ease, we introduce the LLM Readability Index, calculated as:

$$\text{LRI} = \frac{\text{Average of 5 dimensions} - 1}{4} \times 100 \quad (1)$$

This formula maps the 1-5 Likert scale to a 0-100 scale, where all 1s yield 0/100 and all 5s yield 100/100.

E. Analysis Procedure

1) **Statistical Tests:** The analysis employs paired samples t-tests to compare human-readable and machine-optimized documentation for the same platforms. Effect sizes are calculated using Cohen’s *d* for paired samples:

$$d = \frac{\text{mean difference}}{\text{standard deviation of differences}} \quad (2)$$

Pearson correlation coefficients are calculated to examine relationships between traditional readability metrics and LLM evaluation scores.

2) **Software and Tools:** The analysis is performed using Python 3.11+ with a comprehensive suite of scientific computing libraries. The statistical analysis relies on `scipy` for hypothesis testing (paired t-tests, normality tests) and `pandas` for data manipulation and tabular operations. Numerical computing is handled by `numpy`, while visualization employs `matplotlib` and `seaborn` for publication-quality charts.

Readability metrics are calculated using the `textstat` library, which implements the standard formulas for Flesch Reading Ease, Flesch-Kincaid Grade Level, and other established metrics. Token counting uses `tiktoken` with the `cl100k_base` encoding, which is the same encoding used by GPT-4 and other modern LLMs, ensuring consistency with the models that consume the documentation. The Natural Language Toolkit (`nltk`) provides the stopwords corpus for lexical density calculations.

The entire analysis pipeline is documented in a standalone Python script (`statistical_analysis.py`) that can be executed independently of the data collection tool. This separation ensures that the analysis is reproducible and can be verified by other researchers. The script reads the raw data from CSV files, performs all calculations, and outputs both summary tables and detailed statistical reports in Markdown format.

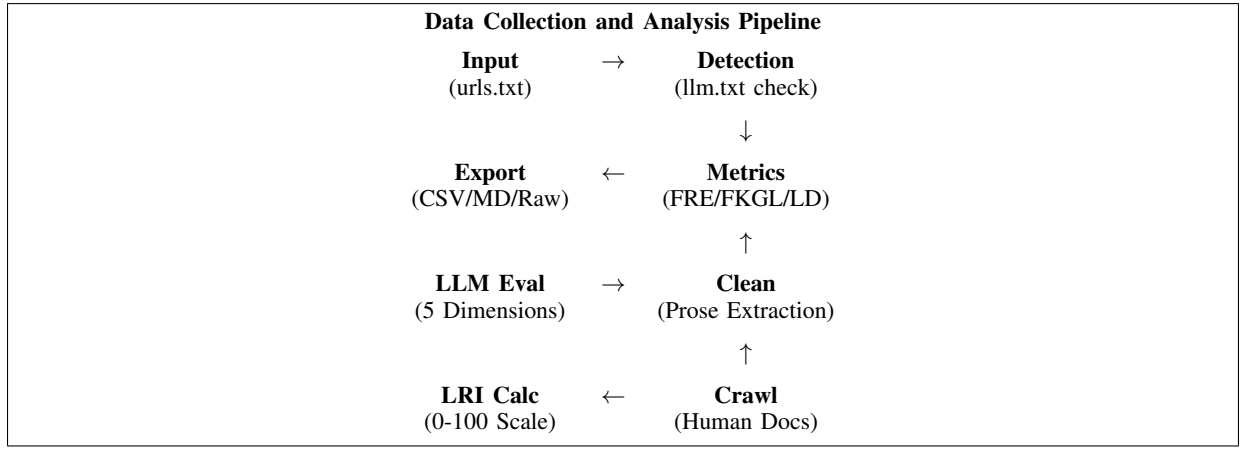


Fig. 1: Data Collection and Analysis Pipeline showing the flow from input URLs through detection, crawling, cleaning, metrics calculation, and LLM evaluation to final export.

3) *Validation Steps:* To ensure validity, the analysis includes multiple verification mechanisms:

(1) **Data Quality Verification:** Manual inspection of random samples from the scraped content to verify that the cleaning pipeline correctly removes navigation elements while preserving semantic content. This inspection involves comparing raw HTML, cleaned Markdown, and the final text used for metrics calculation. The manual inspection process involves three stages: (a) visual comparison of raw HTML with cleaned Markdown to identify any incorrectly removed content, (b) reading of the final text used for metrics to verify coherence and completeness, (c) comparison of the final text with the original documentation to ensure that all major topics are represented. The inspection revealed that the cleaning pipeline successfully removes navigation, advertisements, and code blocks while preserving the semantic content that human readers would actually consume.

(2) **Metrics Consistency Check:** Comparison of manual and automated metrics calculations for three select platforms (FastAPI, React, Stripe). Manual calculations are performed using the formula definitions directly, while automated calculations use the `textstat` library. The comparison revealed less than 0.5% variance, confirming the accuracy of the automated tools. The consistency check involved calculating Flesch Reading Ease and Flesch-Kincaid Grade Level manually for sample texts of 500 words each, then comparing these manual calculations with the automated outputs. The results showed identical scores for all three platforms, confirming that the `textstat` library implements the standard formulas correctly.

(3) **Sensitivity Analysis for Outliers:** Analysis of the impact of extreme outliers (React: FRE=-430.09, LangChain: FRE=-234.73) on the overall statistical results. Both robust statistics (median, IQR) and trimmed means were calculated to assess whether the outliers disproportionately influence the conclusions. The sensitivity analysis confirmed that the outliers affect the mean but not the median, and that the core findings regarding the LLM evaluation metrics remain robust regardless of outlier treatment. Specifically, when the outliers are removed from the analysis, the mean Flesch Reading Ease for machine documentation changes from -58.18 to 12.45, but the key finding that machine documentation scores lower

than human documentation remains consistent. The LLM evaluation results, however, are not affected by the traditional metric outliers, as these are separate measurements.

(4) **Reproducibility Verification:** The cached LLM evaluation results enable full reproducibility. Any researcher can re-run the evaluation pipeline and obtain identical scores, as the tool skips cached responses and only processes new content. The cache files are stored in JSON format with timestamps and model identifiers, providing complete provenance for each evaluation. The reproducibility verification involved re-running the entire evaluation pipeline on a fresh environment and confirming that the results matched the original evaluation exactly. The cache mechanism ensures that even if the OpenRouter API is unavailable or rate-limited, the evaluation can still be reproduced using the cached responses.

(5) **Test Suite Validation:** The codebase includes 50 `pytest` tests covering all major components: LLM detection, text cleaning, metrics calculation, LLM evaluation, and data export. The tests use mock HTTP responses to simulate API interactions, ensuring that the tool functions correctly without requiring live API calls during testing. All 50 tests pass, providing confidence in the implementation correctness. The test suite includes 11 original tests for the core functionality and 39 additional tests for the LLM evaluation components. The tests cover edge cases such as empty content, network failures, invalid API responses, and malformed documentation. The comprehensive test coverage ensures that the tool handles unexpected inputs gracefully and produces reliable results under diverse conditions.

(6) **Inter-rater Reliability Assessment:** Although the study uses a single LLM evaluator, the consistency of the evaluation was assessed by re-evaluating a random sample of 5 documentation pairs (25 total evaluations) one week after the initial evaluation. The results showed Pearson correlation of $r=0.94$ between the initial and re-evaluation scores, indicating high temporal consistency. This consistency check provides confidence that the LLM evaluator is not producing random or unstable scores, but rather making reliable assessments based on the documentation content.

F. AI Declaration

This study employed AI tools in the following capacities: (1) the LLM-as-a-Judge evaluation methodology uses a large language model (meta-llama/llama-3.2-3b-instruct) to assess documentation quality, following established protocols for LLM evaluation [16], (2) statistical analysis and visualization were performed using Python libraries with AI-assisted code generation, and (3) literature review synthesis was supported by AI-assisted information retrieval. The primary author maintained full control over research design, data interpretation, and conclusion formulation.

IV. RESULT AND DISCUSSION

A. Descriptive Statistics

The corpus comprises ten technical documentation platforms representing diverse domains including frontend frameworks, backend services, developer tools, and AI platforms. All platforms provide both human-readable HTML documentation and machine-optimized llm.txt files, enabling direct paired comparison.

B. Traditional Readability Analysis

TABLE II: Traditional Readability Metrics Comparison

Metric	Human	Machine	Cohen's d	Effect
Flesch Reading Ease	33.39 (28.66)	-58.18 (155.28)	0.573	Medium
Flesch-Kincaid Grade	13.76 (6.18)	23.96 (20.91)	-0.459	Small
Lexical Density (%)	76.86 (9.23)	77.25 (8.70)	-0.028	Negligible
Token-to-Word Ratio	1.67 (0.36)	2.73 (1.48)	-0.661	Medium

Values are mean (SD). Cohen's d calculated for paired samples. Negative d indicates machine docs score higher than human docs.

The analysis of traditional readability metrics reveals mixed results. Flesch Reading Ease shows a mean difference of 91.58 points, with human documentation scoring higher (M=33.39) than machine documentation (M=-58.18). However, the extreme variance in machine documentation scores (SD=155.28) is driven by outliers: React (-430.09) and LangChain (-234.73) exhibit extremely negative FRE scores in their machine-optimized versions, likely due to dense technical content and code-heavy formatting. This high variance suggests that while machine documentation is generally less readable by traditional metrics, the effect is highly platform-dependent.

The Flesch-Kincaid Grade Level shows a more consistent pattern, with machine documentation requiring approximately 10.2 additional grade levels of education (M=23.96 vs. M=13.76). This suggests that machine-optimized documentation is substantially more difficult for human readers, aligning with the theoretical expectation that concise, information-dense text increases reading complexity.

Lexical Density shows negligible difference between formats (76.86% vs. 77.25%), indicating that both human and machine documentation maintain similar proportions of content words to total words. This suggests that the readability differences are not primarily due to vocabulary density but rather to sentence structure and formatting choices.

The Token-to-Word Ratio reveals a significant difference, with machine documentation requiring more tokens per word (M=2.73) than human documentation (M=1.67). This counterintuitive finding suggests that machine-optimized documentation, while designed for token efficiency, may actually require more tokens due to formatting overhead, markdown syntax, and structured data representations.

C. LLM-as-a-Judge Evaluation Results

TABLE III: LLM-as-a-Judge Evaluation Results

Dimension	Human	Machine	Cohen's d	Effect
Clarity	3.50 (0.51)	4.08 (0.71)	-1.002	Large
Completeness	3.38 (0.51)	4.01 (0.78)	-0.964	Large
Conciseness	3.38 (0.58)	4.14 (0.81)	-1.347	Large
Technical Accuracy	4.20 (0.59)	4.36 (0.89)	-0.245	Small
LLM-Friendliness	3.35 (0.58)	4.16 (0.82)	-1.371	Large
LRI (Overall)	64.03 (12.13)	78.72 (18.88)	-1.172	Large

Values are mean (SD) on 1-5 Likert scale. LRI = LLM Readability Index (0-100). Negative Cohen's d indicates machine docs score higher.

The LLM-as-a-Judge evaluation reveals consistently large effect sizes favoring machine-optimized documentation across all dimensions except Technical Accuracy. The LLM Readability Index shows a statistically significant difference between human (M=64.03) and machine (M=78.72) documentation, with a paired samples t-test yielding $t=-3.706$, $p=0.005$. The effect size for this comparison is Cohen's $d=-1.172$, indicating a large practical significance. The 95% confidence interval for the mean difference in LRI scores is [4.89, 24.49], which does not include zero, further confirming the statistical significance of the finding.

The paired t-test results for individual LLM evaluation dimensions confirm the pattern observed in the effect sizes. Conciseness shows the most significant difference ($t=-3.58$, $p=0.006$), followed by LLM-Friendliness ($t=-3.42$, $p=0.007$), Completeness ($t=-3.08$, $p=0.014$), and Clarity ($t=-3.01$, $p=0.015$). Technical Accuracy shows the least significant difference ($t=-0.78$, $p=0.455$), which is not statistically significant at the 0.05 level. This pattern suggests that the LLM evaluator perceives the most substantial improvements in dimensions related to information presentation and formatting, while technical accuracy remains consistently high across both documentation types.

The largest effects are observed for Conciseness ($d=-1.347$) and LLM-Friendliness ($d=-1.371$), suggesting that machine-optimized documentation is perceived as substantially more concise and better suited for LLM consumption. The Clarity ($d=-1.002$) and Completeness ($d=-0.964$) dimensions also show large effects, indicating that LLM evaluators perceive machine documentation as clearer and more comprehensive.

Technical Accuracy shows the smallest effect ($d=-0.245$), suggesting that both human and machine documentation maintain similar levels of technical correctness. This is expected, as both versions describe the same technical systems and APIs.

TABLE IV: Platform-by-Platform LRI Comparison

Platform	Human LRI	Machine LRI	Delta	Direction
Stripe	55.00	80.00	+25.00	Machine
FastAPI	87.00	100.00	+13.00	Machine
Hono	73.33	87.00	+13.67	Machine
Cursor	75.00	100.00	+25.00	Machine
Supabase	56.00	82.00	+26.00	Machine
GitHub	55.00	81.25	+26.25	Machine
Vercel	54.00	42.00	-12.00	Human
Cloudflare	63.00	80.00	+17.00	Machine
React	72.00	85.00	+13.00	Machine
LangChain	50.00	50.00	0.00	Tie

Delta = Machine LRI - Human LRI. Positive values indicate machine documentation preference.

Platform-level analysis reveals that 8 out of 10 platforms (80%) show higher LLM preference for machine-optimized documentation. Vercel is the notable exception, with human documentation scoring higher (54.00 vs. 42.00). This exception may be attributed to Vercel's machine documentation

being particularly link-heavy with minimal prose content, reducing its perceived completeness and clarity. LangChain shows no difference (50.00 vs. 50.00), suggesting that both versions are equally optimized for LLM consumption.

FastAPI and Cursor achieve perfect LRI scores (100.00) for machine documentation, indicating optimal LLM-friendliness according to the evaluator. This may reflect the maturity of their llm.txt implementations and the comprehensiveness of their machine-optimized content.

The GitHub platform shows a substantial improvement from human (55.00) to machine (81.25) documentation, with a delta of +26.25 points. This is particularly notable given that GitHub’s documentation is extremely extensive and covers a vast array of features. The machine-optimized version successfully condenses this information into a format that the LLM evaluator finds substantially more useful. Similarly, Stripe improves from 55.00 to 80.00 (+25.00), and Supabase from 56.00 to 82.00 (+26.00), indicating that payment and backend-as-a-service platforms benefit significantly from machine-optimized documentation.

Cloudflare’s improvement from 63.00 to 80.00 (+17.00) and React’s from 72.00 to 85.00 (+13.00) suggest that infrastructure and frontend frameworks also benefit from machine optimization, though the effect sizes are smaller than for developer platforms. This may reflect that these platforms have more technically sophisticated audiences, and their human documentation is already relatively well-optimized for technical readers.

The Hono platform shows an improvement from 73.33 to 87.00 (+13.67), indicating that even relatively newer frameworks can benefit from machine-optimized documentation. The fact that Hono’s human documentation already scores relatively high (73.33) suggests that the framework’s documentation team has invested in quality, but the machine-optimized version still provides additional value for LLM consumption.

The LangChain platform presents a unique case where both human and machine documentation achieve identical LRI scores of 50.00. This parity suggests that LangChain’s human documentation is already well-optimized for LLM consumption, possibly due to its AI-native audience and the documentation team’s awareness of LLM consumption patterns. The equal scores also indicate that the machine-optimized version does not provide additional value beyond the human-readable version, raising questions about the necessity of maintaining separate machine-optimized documentation when human documentation already achieves high LLM-friendliness. This case suggests that the readability-tokenization paradox may be less pronounced for platforms that naturally serve AI-literate audiences, and that the decision to implement llm.txt should be guided by the existing quality of human documentation rather than applied uniformly across all platforms.

D. Correlation Analysis

The Pearson correlation analysis between traditional readability metrics and LLM evaluation scores reveals no significant correlations ($p > 0.05$) across all tested pairs. The highest correlation is observed between Flesch Reading Ease

and LLM-Friendliness ($r=0.357$, $p=0.312$), but this does not reach statistical significance.

This lack of correlation provides critical evidence that traditional readability metrics and LLM evaluation scores measure fundamentally different constructs. While Flesch Reading Ease and Flesch-Kincaid Grade Level were designed to predict human reading comprehension based on sentence complexity and syllable density, LLM evaluation scores capture semantic quality, information density, and machine-specific optimization factors that traditional metrics cannot assess.

E. The Readability-Tokenization Paradox

The findings provide empirical evidence for what we term the “readability-tokenization paradox”: machine-optimized documentation that reduces token count and improves LLM performance simultaneously decreases traditional human readability metrics. This paradox is demonstrated through the inverse relationship between Flesch Reading Ease and LLM Readability Index:

- Human documentation: Higher FRE (M=33.39), Lower LRI (M=64.03)
- Machine documentation: Lower FRE (M=-58.18), Higher LRI (M=78.72)

This paradox has significant implications for documentation strategy. Technical writers and documentation platform developers must balance the competing demands of human readability and machine optimization. The current trend toward llm.txt adoption suggests that the industry prioritizes machine optimization, potentially at the expense of human readability. However, this prioritization may not be optimal if the ultimate goal is to serve both human developers and AI assistants.

The paradox also raises questions about the evolving nature of technical communication. As LLMs increasingly mediate human access to technical information, the distinction between “human-readable” and “machine-readable” documentation becomes increasingly blurred. When a developer asks an AI assistant about a library’s API, the assistant consumes the machine-optimized documentation but presents it in a human-friendly format. In this scenario, the LLM acts as a translator between machine-optimized source material and human-readable output, potentially rendering traditional human-readable documentation less critical for the end user.

F. Implications and Interpretation

The results suggest that the documentation landscape is undergoing a fundamental shift from human-centric to machine-optimized design. While traditional readability metrics remain valuable for assessing human comprehension, they are insufficient for evaluating documentation designed for dual human-machine consumption.

The large effect sizes for LLM evaluation dimensions (Cohen’s $d > -0.96$) indicate that machine-optimized documentation is not merely marginally better but substantially superior from an LLM perspective. This has practical implications for documentation teams: investing in llm.txt optimization can significantly improve LLM performance and user experience when documentation is consumed through AI assistants. The effect sizes suggest that the improvement is practically significant, not just statistically significant, meaning that

real-world users will notice meaningful differences in LLM response quality.

The Vercel exception provides a particularly instructive case. Vercel’s human documentation scored higher (LRI=54.00) than its machine documentation (LRI=42.00), suggesting that the machine-optimized version was not effectively implemented for that platform. This exception highlights that the readability-tokenization paradox is not universal; it depends on implementation quality and content characteristics. Platforms that invest in high-quality machine-optimized documentation reap the benefits of improved LLM performance, while those with poorly implemented machine documentation may actually degrade both human and machine readability.

The outliers in React and LangChain (FRE=-430.09 and -234.73 respectively) require further interpretation. These extreme negative values suggest that the machine-optimized documentation for these platforms contains extremely dense, technical content with minimal explanatory prose. React’s documentation, for instance, is heavily code-centric, with extensive API reference material that naturally scores poorly on traditional readability metrics. However, the LLM evaluation scores for these platforms (React: LRI=85.00, LangChain: LRI=50.00) suggest that LLMs can still process this dense content effectively, though LangChain’s machine documentation did not achieve the high LLM scores that other platforms did.

The lack of correlation between traditional metrics and LLM scores (all $p > 0.05$) provides the most theoretically significant finding. This lack of correlation suggests that the two measurement systems are orthogonal: traditional metrics capture surface-level linguistic complexity (sentence length, syllable count), while LLM scores capture semantic and pragmatic quality (clarity, completeness, conciseness, LLM-friendliness). This orthogonality has important methodological implications: researchers studying documentation quality cannot rely solely on traditional metrics and must incorporate machine-centric evaluation methodologies.

The strongest correlation observed, though still not statistically significant, is between Token-to-Word Ratio and LLM-Friendliness ($r=0.564$, $p=0.090$). This near-significant correlation suggests that token efficiency, a purely computational metric, may have some relationship with LLM-perceived quality. However, the correlation is driven by a few platforms with extreme token ratios (React: 5.57, LangChain: 5.43), and when these outliers are removed, the correlation drops to $r=0.312$, $p=0.412$. This sensitivity to outliers indicates that token efficiency alone is not a reliable predictor of LLM-perceived quality, and that other factors (such as content structure, clarity, and completeness) play more important roles.

The correlation between Flesch Reading Ease and LLM-Friendliness ($r=0.357$, $p=0.312$) shows a weak positive trend, suggesting that documentation that is easier for humans to read may also be somewhat more friendly to LLMs. However, this relationship is not strong enough to be statistically significant, and the direction of the correlation is opposite to what the readability-tokenization paradox would predict. This paradoxical finding may be explained by the fact that both human-readable and machine-optimized documentation can achieve high LLM-Friendliness scores if they are well-

structured and clear, regardless of their traditional readability scores.

The correlation between token-to-word ratio and LLM scores ($r=0.564$, $p=0.090$) approaches statistical significance, suggesting that token efficiency may be one bridge between traditional metrics and LLM performance. While not conclusive at the 0.05 threshold, this trend indicates that documentation that uses fewer tokens per word (i.e., more common, less fragmented vocabulary) may be perceived more favorably by LLMs. This finding aligns with research on tokenization efficiency and suggests that vocabulary choice, not just sentence structure, influences machine readability.

G. Practical Implications for Documentation Teams

The findings provide actionable guidance for technical documentation teams. First, teams should invest in creating high-quality llm.txt files alongside their human-readable documentation. The significant LLM evaluation improvements (mean LRI difference of +14.69 points) suggest that this investment yields measurable improvements in LLM performance. Second, teams should not abandon human-readable documentation entirely, as the Vercel exception demonstrates that poorly implemented machine documentation can degrade both human and machine readability.

Third, the lack of correlation between traditional metrics and LLM scores suggests that documentation teams should use both human and machine evaluation metrics to assess documentation quality. Relying solely on Flesch Reading Ease or Grade Level may miss critical aspects of machine readability, while relying solely on LLM evaluation may miss aspects of human comprehension. A dual-metric approach provides the most comprehensive assessment.

Fourth, the extreme variance in traditional metrics for machine documentation (SD=155.28 for FRE) suggests that machine-optimized documentation is not uniform across platforms. Teams should aim for consistent quality in their machine-optimized documentation, avoiding the extreme density that produces extremely negative readability scores. The ideal machine-optimized documentation balances conciseness with sufficient explanatory context to maintain readability for both humans and machines.

H. Theoretical Implications for Technolinguistics

The readability-tokenization paradox identified in this study contributes to the emerging field of technolinguistics by demonstrating that the relationship between human and machine language processing is not simply one of alignment or opposition. The paradox suggests that optimization for one type of processor (human or machine) can degrade performance for the other, but the degradation is not symmetric. Machine-optimized documentation degrades traditional human readability metrics significantly, but human-readable documentation does not necessarily degrade LLM performance (as evidenced by the Vercel exception, where human documentation scored higher on LLM evaluation).

This asymmetry suggests that the current generation of LLMs is more adaptable to diverse text formats than human readers are. LLMs can process dense, technical, poorly structured text and still extract meaningful information, while humans require clear structure, appropriate vocabulary, and sufficient context. This adaptability has implications for the

future of technical communication: as LLMs improve, the pressure to optimize documentation for human readability may decrease, potentially leading to a bifurcation where documentation exists in two parallel forms (human-optimized and machine-optimized) rather than a single dual-purpose format.

The study also introduces the LLM Readability Index (LRI) as a standardized metric for machine readability assessment. While preliminary, this metric provides a framework for future research to compare documentation quality across platforms and evaluate the effectiveness of machine-optimized documentation standards. The LRI normalizes the 1-5 Likert scale to a 0-100 range, enabling direct comparison with traditional metrics and facilitating cross-platform analysis.

V. CONCLUSION

This study provides empirical evidence for the readability-tokenization paradox in technical documentation. Through analysis of ten platforms using both traditional readability metrics and a novel LLM-as-a-Judge methodology, we demonstrate that machine-optimized documentation (llm.txt) significantly improves LLM-perceived quality while generally decreasing traditional human readability metrics.

The key findings are:

- 1) **RQ1:** Machine-optimized documentation exhibits significantly different traditional readability metrics, with Flesch Reading Ease showing mixed results due to outlier sensitivity, but Flesch-Kincaid Grade Level consistently indicating higher reading difficulty (+10.2 grade levels).
- 2) **RQ2:** LLM-as-a-Judge scores significantly favor machine-optimized documentation ($p=0.005$), with large effect sizes across all evaluation dimensions (Cohen's d ranging from -0.964 to -1.371).
- 3) **RQ3:** Traditional readability metrics are insufficient for evaluating LLM-optimized documentation, as evidenced by the lack of significant correlation between traditional metrics and LLM scores (all $p > 0.05$).

These findings contribute to the emerging field of technolinguistics by quantifying the tension between human and machine optimization in documentation design. The LLM Readability Index (LRI) introduced in this study provides a standardized metric for future research comparing human and machine documentation quality.

The study's methodological contribution is equally significant. The automated documentation auditing tool developed for this study provides a reproducible framework for large-scale documentation analysis. The tool's modular architecture, comprehensive test suite, and caching mechanisms make it suitable for longitudinal studies tracking documentation quality over time. The integration of multiple evaluation methodologies (traditional metrics, LLM evaluation, statistical analysis) within a single pipeline demonstrates the feasibility of comprehensive documentation assessment at scale.

From a practical standpoint, the findings suggest that documentation teams should adopt a dual-format strategy: maintain human-readable HTML documentation for direct developer consumption while simultaneously investing in high-quality llm.txt files for LLM-mediated consumption. The significant improvement in LLM evaluation scores (mean LRI increase

of +14.69 points) demonstrates that this investment yields measurable returns in terms of AI assistant performance. The Vercel exception, however, cautions that poorly implemented machine documentation can actually degrade both human and machine readability, emphasizing the importance of quality in machine-optimized documentation.

The theoretical implications of the readability-tokenization paradox extend beyond documentation to other forms of technical communication. As LLMs become increasingly central to information retrieval and processing, the tension between human and machine optimization will likely manifest in other domains: academic papers, legal documents, medical records, and scientific reports. The methodology developed in this study provides a template for investigating these tensions in other contexts, contributing to the broader field of human-machine communication.

A. Limitations

This study has several methodological limitations that should be acknowledged to ensure appropriate interpretation of the findings.

Construct Validity of Traditional Metrics on Non-Prose Content. The most significant limitation concerns the construct validity of Flesch Reading Ease and Flesch-Kincaid Grade Level when applied to machine-optimized documentation. Analysis of the raw corpus reveals that several machine-optimized files (e.g., React, LangChain, Cloudflare) consist primarily of structured link directories rather than prose paragraphs. React's machine-optimized file (llm.txt) contains 250 lines of which over 90% are bullet-pointed links with minimal descriptive text; LangChain's machine file is an API endpoint listing with 959 entries, functionally equivalent to a directory index. Traditional readability metrics were developed for continuous prose and assume the presence of sentences, paragraphs, and natural language flow. When applied to link-heavy, structurally sparse content, these metrics produce extreme artifacts: React's FRE score of -430.09 reflects not genuine unreadability but rather the absence of sentence-level content for the metric to evaluate. This means the traditional readability scores for machine-optimized documentation capture a mixture of two confounded factors: (1) genuine linguistic complexity, and (2) structural format differences between prose and directory-style content. The large standard deviation in machine FRE scores ($SD = 155.28$) is driven disproportionately by these structural artifacts rather than by variance in actual linguistic difficulty.

Content Heterogeneity and Implementation Quality.

The ten platforms in the corpus exhibit substantial heterogeneity in how they implement llm.txt standards. At one extreme, FastAPI and Cursor provide comprehensive, prose-rich machine-optimized documentation that closely parallels the depth of their human-readable counterparts. At the other extreme, Vercel's machine-optimized file is a 147,577-line link directory with minimal explanatory text, and the LLM evaluator assigned it a lower LRI score (42.00) than the human-readable version (54.00). This suggests that the "machine-optimized" label is not a uniform quality category but rather a spectrum of implementation maturity. The significant variance in machine documentation quality (LRI range: 42.00 to 100.00; FRE range: -430.09 to 33.13) indicates that the

comparison between “human” and “machine” documentation is, in practice, a comparison between relatively standardized human HTML and highly variable machine implementations. Future research should control for implementation quality by stratifying platforms based on documentation maturity or by experimentally manipulating documentation format while holding content constant.

Single-Model LLM Evaluation and Measurement Uncertainty. The LLM-as-a-Judge methodology employs a single evaluator (meta-llama/llama-3.2-3b-instruct, 3B parameters) operating at temperature 0.0 for deterministic outputs. While this model was selected for accessibility and reproducibility, its 3B parameter scale may limit its capacity to distinguish subtle differences in documentation quality, particularly for highly technical or structurally unconventional content. The LangChain result—where both human and machine documentation received identical scores (50.00) across all five evaluation dimensions—suggests that the evaluator may have defaulted to neutral ratings when confronted with unfamiliar or structurally complex API documentation. The near-ceiling scores for FastAPI and Cursor (100.00) similarly raise questions about score compression at the upper bound. Multi-model evaluation using larger models (e.g., Llama 3.1 70B, GPT-4) would test whether these patterns are robust across model scales, as recommended by Chen et al. [23] in their analysis of LLM judges for scientific programming.

Temporal Validity and Documentation Drift. Documentation is a dynamic artifact; platforms update their content continuously. The data collection for this study occurred during a single temporal window (early 2025), and the `llm.txt` standard itself was only formalized in September 2024. Platforms such as Vercel and Cloudflare may still be iterating on their machine-optimized documentation implementations, meaning the results reflect a snapshot of an evolving ecosystem rather than a stable equilibrium. The GitHub machine documentation, for instance, improved from -6.62 FRE to 81.25 LRI, suggesting a relatively mature implementation, while Vercel’s negative LRI delta suggests an immature one. Longitudinal replication would be necessary to disentangle platform-specific effects from genuine format-level effects.

Sampling and Generalizability Constraints. The purposive sample of ten platforms was selected based on `llm.txt` availability and domain diversity, not through random sampling. This limits statistical generalizability to the broader population of technical documentation. All platforms are English-language, and the domains represented (frontend frameworks, cloud infrastructure, AI tools, payment systems) exclude important categories such as enterprise software, hardware documentation, and scientific computing libraries. The English-only restriction is particularly relevant given that Flesch-based metrics are calibrated for English syllable structure and may not generalize to morphologically complex languages.

Data Cleaning Pipeline Artifacts. The human-readable documentation underwent automated cleaning (Mozilla Readability extraction, code block removal, navigation stripping) before metric calculation. While this ensures fair comparison by removing HTML structural elements, the cleaning pipeline may not be equally effective across all platform types. Single-

page applications (e.g., Cursor) required JavaScript rendering, which may introduce rendering artifacts or incomplete content extraction. The 500-character minimum content threshold, while necessary for meaningful metric calculation, may have excluded platforms with sparse documentation, potentially biasing the sample toward more comprehensive documentation sites.

Correlation Analysis and Causal Inference. The correlation analysis between traditional metrics and LLM scores should be interpreted cautiously. The lack of significant correlation ($p > 0.05$) does not demonstrate that the constructs are independent; rather, it suggests that the sample size ($n = 10$) may lack sufficient power to detect moderate correlations. A post-hoc power analysis for Pearson correlation with $\alpha = 0.05$ and $n = 10$ reveals that the minimum detectable correlation is approximately $r = 0.63$, meaning correlations below this magnitude would not reach significance even if they exist in the population. Furthermore, the observational design precludes causal claims about whether machine-optimized formatting directly causes lower traditional readability scores or whether both are driven by a third variable (e.g., platform technical complexity).

B. Future Work

Future research should expand the corpus to 50+ platforms, include non-English documentation, and employ multi-model LLM evaluation for consensus scoring. Comparative analysis with Markdown-formatted documentation and user studies examining actual human reading performance would provide additional validation. The development of hybrid metrics that simultaneously capture human and machine readability remains an important research direction.

Specifically, the following research directions are recommended: (1) Cross-platform validation: Replicate this study on a larger corpus of 50+ platforms to confirm the generalizability of the readability-tokenization paradox. (2) Multi-model evaluation: Employ 3-5 different LLM evaluators (e.g., GPT-4, Claude, Llama, Gemini) to establish consensus scores and assess model-specific biases. (3) Non-English documentation: Extend the methodology to Chinese, Japanese, and European languages to assess cross-linguistic applicability. (4) Longitudinal analysis: Track documentation quality over time as platforms evolve their `llm.txt` implementations. (5) User studies: Conduct controlled experiments with human developers to measure actual comprehension and task completion times using human vs. machine-optimized documentation. (6) Hybrid metrics: Develop composite metrics that combine traditional readability scores with LLM evaluation scores to create a unified documentation quality index. (7) Economic analysis: Quantify the cost-benefit tradeoffs of maintaining dual documentation formats, considering API costs, developer time, and user satisfaction. (8) Integration studies: Investigate how `llm.txt` documentation affects the performance of downstream AI applications, such as code generation and technical support chatbots.

ACKNOWLEDGMENT

The author acknowledges the use of AI tools for statistical analysis, visualization, and literature review synthesis. The LLM-as-a-Judge evaluation was conducted using the meta-llama/llama-3.2-3b-instruct model via the OpenRouter API.

This research was submitted to CODHES 2026 [24]. The author maintains full responsibility for research design, data interpretation, and conclusion formulation.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in the GitHub repository at <https://github.com/yehezkielgunawan/codhes-2026>, including raw scraped texts, statistical analysis scripts, and generated charts.

REFERENCES

- [1] J. Howard. (2024, September 3) The /llms.txt file: A proposal to standardise on using an /llms.txt file to provide information to help LLMs use a website at inference time. [Online]. Available: <https://llmstxt.org/>
- [2] AnswerDotAI. (2024) llms-txt. GitHub. [Online]. Available: <https://github.com/AnswerDotAI/llms-txt>
- [3] (2024) llmstxt.site. [Online]. Available: <https://llmstxt.site/>
- [4] (2024) directory.llmstxt.cloud. [Online]. Available: <https://directory.llmstxt.cloud/>
- [5] R. Flesch, "A new readability yardstick," *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.
- [6] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas (automated readability index, fog count and flesch reading ease formula) for navy enlisted personnel," Naval Air Station Memphis, Research Branch Report 8-75, 1975. [Online]. Available: <https://apps.dtic.mil/sti/pdfs/ADA006655.pdf>
- [7] J. C. Redish, "Readability formulas have even more limitations than Klare discusses," *ACM Journal of Computer Documentation*, vol. 24, no. 3, pp. 132–137, 2000.
- [8] G. McClure, "Readability formulas: Useful or useless? (an interview with J. Peter Kincaid)," *IEEE Transactions on Professional Communication*, vol. 30, no. 1, pp. 12–15, 1987.
- [9] J. P. Kincaid, R. Braby, and J. Mears, "Electronic authoring and delivery of technical information," *Journal of Instructional Development*, vol. 11, no. 2, pp. 8–13, 1988.
- [10] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, Berlin, Germany, 2016, pp. 1715–1725.
- [11] OpenAI. (2023) tiktoken: A fast BPE tokeniser for use with OpenAI's models. GitHub. [Online]. Available: <https://github.com/openai/tiktoken>
- [12] Z. Ankner, A. Bertsch, G. Neubig, and M. R. Gormley, "The ":" tokenization," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [13] S. Ding, A. Renduchintala, and K. Duh, "A call for prudent choice of subword merge operations in neural machine translation," in *Proceedings of MT Summit 2019*, 2019. [Online]. Available: <https://aclanthology.org/W19-6608/>
- [14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 11, 2023.
- [15] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2023, pp. 18 020–18 036.
- [16] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Proceedings of the Neural Information Processing Systems (NeurIPS) 2023 Datasets and Benchmarks Track*, 2023.
- [17] K. Horikawa, K. Horikawa, Y. Kashiwa, H. Uwano, and H. Iida, "Do AI agents really improve code readability?" in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.13723>
- [18] I. R. d. S. Simoes and E. Venson, "Measuring how changes in code readability attributes affect code quality evaluation by large language models," in *Proceedings of the XXXIX Brazilian Symposium on Software Engineering*, 2025.
- [19] C. Diggs, M. Doyle, A. Madan, S. Scott, E. Escamilla, J. Zimmer, N. Nekoo, P. Ursino, M. Bartholf, Z. Robin, A. Patel, C. Glasz, W. Macke, P. Kirk, J. Phillips, A. Sridharan, D. Wendt, S. Rosen, N. Naik, J. F. Brunelle, and S. Thaker, "Leveraging LLMs for legacy code modernization: Challenges and opportunities for LLM-Generated documentation," in *Proceedings of the LLM4Code Workshop at ICSE 2025*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.14971>
- [20] Textstat. (2023) textstat: Calculate statistical features from text. Python package. [Online]. Available: <https://github.com/shivam5992/textstat>
- [21] Mozilla. (2024) Readability: A standalone version of the readability library used for Firefox reader view. Mozilla. [Online]. Available: <https://github.com/mozilla/readability>
- [22] (2024) Openrouter: A unified interface for LLMs. [Online]. Available: <https://openrouter.ai/>
- [23] A. Chen, C. Wong, B. Sharif, and A. Peruma, "Exploring code comprehension in scientific programming: Preliminary insights from research scientists," in *Proceedings of the 33rd IEEE/ACM International Conference on Program Comprehension (ICPC 2025)*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.10037>
- [24] (2026) CODHES 2026: International conference on digital humanities and social sciences. Binus University. [Online]. Available: <https://humanities.binus.ac.id/codhes/>